

day04笔记

一、面向对象特点

1、封装

- 把数据和操作数据的方法放到一个类里面，并隐藏内部细节
- 开放部分权限给别人使用，有利于数据的安全

```
# username = '坤坤'
# def demo_fn():
#     # username = '坤坤'
#     print(username)
#
# demo_fn()
#
# print(username)

# 命名污染（变量名冲突） ---- 全局变量，在任意位置都可以进行访问
# username = '张三'
#
# print(username)

def demo_fn():
    # 局部变量，只能在当前函数内部进行访问，其他地方访问不了的（存在作用域）
    # 封装
    username = '坤坤'
    print(username)

demo_fn()

# print(username) # name 'username' is not defined

def demo_fn2():
    # 局部变量，只能在当前函数内部进行访问，其他地方访问不了的（存在作用域）
    # 封装
    username = '张三'
    print(username)

demo_fn2()

def say():
    print('张三说，要努力对方向！')

# 注意点：同名函数，也会有冲突问题
def say():
    print('坤坤说，只要粉丝爱我就行！')
```

say()

2、继承

- 子类可以继承父类的属性和方法
- 进行代码的复用，减少冗余代码，提高开发效率

3、多态

- 同一个动作，不同的表现
- 同一个方法，在不同的对象上有不同的表现

二、已有数据类型问题

1、基础类型

数值类型（int、float）、文本（字符串）类型（str）、布尔类型（True、False）、None（空值类型）

问题：一个变量只能存储一个值，一个变量名称对应一个值

2、容器类型

列表、字典、元组、集合。一个变量可以存储多个值

问题：容器类型只能存储数据，其他操作逻辑需要使用函数来实现

例如：创建一个人的信息

```
person1 = {  
    'name': '丁致字',  
    'age': 21,  
    'gender': '男',  
    'like': ['美女', '篮球', 'rap']  
}  
  
person2 = {  
    'name': '陈乐',  
    'age': 21,  
    'gender': '男',  
    'like': ['美女', '篮球', 'rap']  
}
```

问题：人的基本信息代码都是一样的，创建一个人的信息就需要把这些代码重新写一遍，导致冗余代码太多了

三、创建对象

1、对象数据类型特点

- 属性：用于描述一个物体的特征
- 方法：一个物体的行为
- 对象也是一个可以存储多个数据的容器

2、创建步骤

```

# class 创建对象的参照的模板
class Person:
    # 构造函数（初始化的方法）
    # self --- 指的是创建出来的那个对象，self的指向会改变
    # name='张三', age=18, gender='男' 函数默认参数
    def __init__(self, name='张三', age=18, gender='男'):
        # name、age、gender 都是对象的属性（属性名和属性值）
        self.name = name
        self.age = age
        self.gender = gender

    # 方法（行为）
    def study(self):
        print(f'{self.name} 正在努力的学习! ')

# 实例化对象
p1 = Person('丁致宇', 21, '男')
# 注意点：对象类型，如果使用属性和方法，可以通过对象名称点上属性名或者方法名
print(p1.name)
print(p1.age)
p1.study()

print('-' * 20)

p2 = Person('陈乐', 20, '男')
print(p2.name)
print(p2.age)
p2.study()

```

说明：

- `__init__()` 是类的构造函数，用于初始化对象的属性，当创建实例时，python会自动去调用这个方法
- `self` 是类实例的引用，代表的是当前对象自己，必须作为第一个参数传递

四、对象属性和方法

1、实例属性和实例方法

实例属性

- 是属于某个对象具体的数据，在 `__init__()` 函数中定义
- 每个对象都有自己的一份属性，修改一个对象的属性，不会影响到其他的对象

示例代码

```

# class 创建对象的参照的模板
class Person:
    # 构造函数（初始化的方法）
    def __init__(self, name, age, gender):
        # 实例属性
        self.name = name
        self.age = age

```

```

        self.gender = gender

    # 实例方法
    def study(self):
        print(f'{self.name} 正在努力的学习! ')

p1 = Person('丁致宇', 18, '男')
p2 = Person('陈乐', 19, '男')

p1.name = '丁小宇'
print(p1.name)

print(p2.name)

```

2、类属性和方法

类属性：

- 类的属性是定义在类的内部，但在方法外部的变量。它属于类本身，不属于实例对象
- 可以共享数据，所有实例共用一份数据
- 节省内存空间

示例代码

```

# class 创建对象的参照的模板
class Person:
    # 现在需要统计创建了多少个对象
    counter = 0
    desc = '这是一个创建人的类'
    # 构造函数（初始化的方法）
    def __init__(self, name, age, gender):
        # 实例属性
        self.name = name
        self.age = age
        self.gender = gender

        Person.counter += 1

    # 实例方法
    def study(self):
        print(f'{self.name} 正在努力的学习! ')

    # 定义一个方法来统计，并把结果给返回
    def count_fn(self):
        return Person.counter

p1 = Person('丁致宇', 18, '男')
p2 = Person('陈乐', 19, '男')
p3 = Person('张三', 20, '男')

print(p1.count_fn())

print(Person.counter)

```

```
print(Person.desc)
```

类方法：

- 类方法需要使用 `@classmethod` 装饰器定义的方法
- 第一次参数是 `cls`, 表示类本身, 而不是实例 (`self`)
- 类方法可以在没有实例的时候, 更方便和灵活

示例代码

```
# 第一个特点: 可以在没有实例的情况下使用
class Person:
    price = 100
    def __init__(self, name):
        self.name = name

    @classmethod
    def add_fn(cls, x, y):
        print('使用类属性', cls.price)
        print(x + y)
        return x + y

    def study(self, x, y):
        Person.add_fn(x, y)
        print(f'{self.name} 正在努力的学习! ')

    def say(self, x, y):
        Person.add_fn(x, y)
        print(f'{self.name} 喜欢演讲! ')

p1 = Person('丁致宇')

# print(Person.add_fn(10, 20))

p1.study(10, 20)
p1.say(66, 88)
```

3、静态方法

注意点：在python中并没有静态属性这个专业的术语，但有类属性，这个属性常常被误认为是静态属性

静态方法：

- 使用 `@staticmethod` 装饰器定义
- 没有 `self`、`cls`, 它就是一个放在类里面的普通函数
- 不能访问实例属性, 也不能访问类属性

示例代码：

```
class MathUtils:
    # 静态方法
    @staticmethod
```

```

def add(x, y):
    return x + y

# 静态方法
@staticmethod
def is_even(n):
    return n % 2 == 0

print(MathUtils.add(10, 20))
print(MathUtils.is_even(3))
print(MathUtils.is_even(6))

class User:
    def __init__(self, name, email):
        # 使用验证函数
        if not User.is_email(email):
            print('邮箱格式不正确')
            return

        self.name = name
        self.email = email

    # 验证邮箱格式
    @staticmethod
    def is_email(email):
        return '@' in email and '.' in email

u1 = User('张三', 'abc@qq.com')
print(u1.email)

```

五、继承

1、继承，允许一个类去继承另一个类（父类）的属性和方法，从而实现代码的复用和扩展

2、单继承

单继承指的是，子类只能继承一个父类的属性和方法

```

# 父类
# class Father(object):
#     pass

class Father:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def money(self):
        print('一个小目标')

# 子类
class Son(Father):

```

```
def love(self):
    print('恋爱')

s1 = Son('丁小宇', 18)
print(s1.name, s1.age)
s1.money()
```

3、多继承

多继承指的是一个子类可以继承多个父类的属性和方法

```
# 多继承指的是一个子类可以继承多个父类的属性和方法
# 注意点：如果父类中有相同的属性和方法，默认会使用第一个父类中的属性和方法

class Father:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def money(self):
        print('一个小目标')

class Uncle:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def house(self):
        print('三套房')

    def money(self):
        print('2个小目标')

class Son(Father, Uncle):
    def love(self):
        print('恋爱')

s1 = Son('丁小宇', 20)
s1.money()
s1.house()
print(s1.name)
```

4、子类可以重写父类的属性和方法

```
class Father:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def money(self):
        print('一个小目标')
```

```

class Son(Father):
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    def love(self):
        print('恋爱')

    # 重写父类中的方法
    def money(self):
        print('10个目标')

s1 = Son('丁小宇', 20, '男')
s1.money()
print(s1.name)
print(s1.gender)

```

5、super函数

super函数可以用于调用父类中的方法

```

class Father:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def money(self):
        print('一个小目标')

    def add(self, x, y):
        return x + y

class Son(Father):
    def love(self):
        print('恋爱')

    def calc(self, x, y):
        return super().add(x, y)

s1 = Son('丁小宇', 20)
s1.money()
print(s1.name)

print(s1.calc(10, 20))

```

super函数用的最多的是在继承父类的属性和方法的同时，去扩展自己的属性和方法


```

# super函数用的最多的是在继承父类的属性和方法的同时，去扩展自己的属性和方法
class Father:
    # 第五步：执行 __init__() 函数，并进行形参赋值 name='丁小宇' age=20
    def __init__(self, name, age):
        # 第六步：给对象的属性进行实际的赋值操作
        self.name = name
        self.age = age

    def money(self):
        print('一个小目标')

# 第二步：把父类中的属性和方法，继承过来放在子类这里
class Son(Father):
    # 第三步：由于实例化对象的时候，__init__() 函数已经被调用，这里做的是形参赋值操作 name='丁小宇'
    age=20 gender='男'
    def __init__(self, name, age, gender):
        # 第四步：super() 调用父类中的 __init__() 方法，并进行实参传递
        super().__init__(name, age)
        # 第七步：子类自己的属性进行实际的赋值操作
        self.gender = gender

# 第一步：s1 --- 实例化对象，进行实参的传递
s1 = Son('丁小宇', 20, '男')
s1.money()
print(s1.name, s1.gender)

```

六、作业

1、类继承练习 --- 人力系统

要求：

- 员工分为两类：全职员工 FullTimeEmployee、兼职员工 PartTimeEmployee。
- 全职和兼职都有"姓名 name"、“工号id”属性，
- 都具备"打印信息 print_info"（打印姓名、工号）方法。
- 全职有"月薪 monthly_salary"属性，
- 兼职有"日薪 daily_salary"属性、“每月工作天数 work_days"的属性。
- 全职和兼职都有"计算月薪 calculate_monthly_pay"的方法，但具体计算过程不一样。

2、预习 HTML语言 和 CSS语言